

ClassNest - UNIT-III Assessment Paper

Collections and Object Oriented Programming

Topics: Lists, Tuples, Sets, Dictionary, Python Classes and Objects

Question Pattern: 30 MCQs, 20 Fill-ups, 10 Simple Coding Questions | Total: 100 Marks

Instructions

1. Answer all questions.
2. MCQs carry 1 mark each.
3. Fill-ups carry 1 mark each.
4. Coding questions carry 5 marks each.
5. This student paper does not contain answer keys.

Section A - Multiple Choice Questions (30 x 1 = 30 Marks)

1. A list in Python is best described as:

- A. An ordered collection enclosed in square brackets
- B. An unordered collection of unique values
- C. A key-value collection
- D. A class template

2. Each value stored inside a list is called:

- A. Method
- B. Element
- C. Object
- D. Constructor

3. Python list indexing starts from:

- A. 1
- B. -1
- C. 0
- D. n

4. For marks = [95, 97, 100], what is marks[-1]?

- A. 95
- B. 97
- C. 100
- D. -1

5. Which list method adds a single element at the end?

- A. extend()
- B. append()
- C. insert()
- D. clear()

6. Which list method adds more than one element from another sequence?

- A. append()
- B. pop()
- C. extend()
- D. remove()

7. Which list method inserts an element at a specific position?

- A. insert()
- B. append()
- C. clear()
- D. len()

8. Which method deletes all elements from a list but keeps the list object?

- A. del list
- B. remove()
- C. clear()
- D. pop(0)

9. A tuple is different from a list mainly because tuple elements are:

- A. Mutable
- B. Immutable
- C. Always strings
- D. Always numbers

10. Tuple values are commonly enclosed using:

- A. Square brackets []
- B. Parentheses ()
- C. Curly braces {}
- D. Angle brackets <>

11. Which statement about tuples is correct?

- A. Tuples are faster than lists for fixed data
- B. Tuples must contain only integers
- C. Tuples cannot be indexed
- D. Tuples use key-value pairs

12. What is the output of len((10, 20, 30))?

- A. 2
- B. 3
- C. 4
- D. Error

13. Which operation is invalid for a tuple t = (1,2,3)?

- A. t[0]
- B. len(t)
- C. t[1] = 5
- D. t[-1]

14. A set in Python is:

- A. Ordered and allows duplicates
- B. Unordered and does not allow duplicates
- C. A sequence of characters
- D. A method inside a class

15. Which brackets are commonly used for set literals?

- A. []
- B. ()
- C. {}
- D. <>

16. Which operation combines elements from two sets without duplicates?

- A. Intersection
- B. Difference
- C. Union
- D. Indexing

17. Which set operation gives only elements common to both sets?

- A. Union
- B. Intersection
- C. Difference
- D. Concatenation

18. Which of these is not supported by a set?

- A. Adding an element
- B. Removing an element
- C. Membership testing
- D. Index-based access

19. A dictionary stores data as:

- A. Only numbers
- B. Key-value pairs
- C. Only strings
- D. Indexed characters

20. Dictionary values are accessed using:

- A. Index number only
- B. Key
- C. Line number
- D. Memory address

21. Which symbol separates a key and value in a dictionary?

- A. =
- B. ;
- C. :
- D. ,

22. Which method returns dictionary keys?

- A. values()
- B. items()
- C. keys()
- D. append()

23. Which method returns key-value pairs from a dictionary?

- A. items()
- B. keys()
- C. insert()
- D. popitem(index)

24. Python is considered an object-oriented programming language because it supports:

- A. Classes and objects
- B. Only loops
- C. Only print statements
- D. Only tuples

25. A class is best described as:

- A. An instance of an object
- B. A blueprint or main building block for objects
- C. A loop statement
- D. An input function

26. An object is also called:

- A. Loop
- B. Instance of a class
- C. Keyword
- D. String index

27. Which keyword is used to define a class in Python?

- A. def
- B. object
- C. class

D. init

28. Functions defined inside a class are called:

- A. Members
- B. Methods
- C. Constructors only
- D. Indexes

29. Class members are accessed using which operator?

- A. Comma
- B. Dot
- C. Colon
- D. Hash

30. In Python, the constructor method is:

- A. __init__
- B. __del__
- C. main()
- D. class()

Section B - Fill in the Blanks (20 x 1 = 20 Marks)

31. A list is an ordered collection enclosed by _____ brackets.

Answer: _____

32. The position number of an element in a list is called an _____.

Answer: _____

33. The function used to find the number of elements in a list is _____.

Answer: _____

34. The list method used to add one element at the end is _____.

Answer: _____

35. The list method used to remove all elements but retain the list is _____.

Answer: _____

36. Negative indexing starts from the _____ of the list.

Answer: _____

37. Tuple elements are _____, so they cannot be changed.

Answer: _____

38. Tuple values are usually enclosed within _____.

Answer: _____

39. A tuple with fixed data is preferred when values should not be _____.

Answer: _____

40. A set stores only _____ elements.

Answer: _____

41. The set operation that returns common elements is _____.

Answer: _____

42. The set operation that combines all unique elements is _____.

Answer: _____

43. A dictionary stores data in _____ pairs.

Answer: _____

44. The dictionary method used to get all keys is _____.

Answer: _____

45. The dictionary method used to get all values is _____.

Answer: _____

46. A class is defined using the keyword _____.

Answer: _____

47. An object is an _____ of a class.

Answer: _____

48. Functions inside a class are called _____.

Answer: _____

49. The first argument of an instance method is commonly named _____.

Answer: _____

50. The constructor in Python is written as _____.

Answer: _____

Section C - Simple Coding Questions (10 x 5 = 50 Marks)

51. Write a function `list_sum(nums)` that returns the sum of all numbers in a list. (5 Marks)

Starter code:

```
def list_sum(nums):  
    # write your code here  
    pass
```

Visible test cases:

Input: `list_sum([1, 2, 3])`

Expected: 6

Input: `list_sum([10, -2, 5])`

Expected: 13

52. Write a function `max_mark(marks)` that returns the highest mark from a list. (5 Marks)

Starter code:

```
def max_mark(marks):  
    # write your code here  
    pass
```

Visible test cases:

Input: `max_mark([95, 97, 100])`

Expected: 100

Input: `max_mark([40, 55, 50])`

Expected: 55

53. Write a function `reverse_list(items)` that returns the list in reverse order without changing the original list. (5 Marks)

Starter code:

```
def reverse_list(items):  
    # write your code here  
    pass
```

Visible test cases:

Input: `reverse_list([1, 2, 3])`

Expected: [3, 2, 1]

Input: `reverse_list(['a', 'b'])`

Expected: ['b', 'a']

54. Write a function `tuple_first_last(t)` that returns a tuple containing the first and last elements of t. (5 Marks)

Starter code:

```
def tuple_first_last(t):  
    # write your code here  
    pass
```

Visible test cases:

Input: `tuple_first_last((10, 20, 30))`

Expected: (10, 30)

Input: `tuple_first_last(('x', 'y'))`

Expected: ('x', 'y')

55. Write a function `unique_count(items)` that returns the number of unique elements in a list. (5 Marks)

Starter code:

```
def unique_count(items):  
    # write your code here  
    pass
```

Visible test cases:

Input: `unique_count([1, 1, 2, 3])`

Expected: 3

Input: `unique_count(['a', 'a', 'b'])`

Expected: 2

56. Write a function common_elements(a, b) that returns a sorted list of common elements from two lists. (5 Marks)

Starter code:

```
def common_elements(a, b):  
    # write your code here  
    pass
```

Visible test cases:

Input: common_elements([1,2,3], [2,3,4])
Expected: [2, 3]

Input: common_elements(['a','b'], ['b','c'])
Expected: ['b']

57. Write a function get_student_mark(marks, name) that returns the mark for name from a dictionary. Return 'Not found' if the name is absent. (5 Marks)

Starter code:

```
def get_student_mark(marks, name):  
    # write your code here  
    pass
```

Visible test cases:

Input: get_student_mark({'Arun':90, 'Bala':85}, 'Bala')
Expected: 85

Input: get_student_mark({'Arun':90}, 'Chitra')
Expected: Not found

58. Write a function word_frequency(words) that returns a dictionary with each word and its count. (5 Marks)

Starter code:

```
def word_frequency(words):  
    # write your code here  
    pass
```

Visible test cases:

Input: word_frequency(['py', 'java', 'py'])
Expected: {'py': 2, 'java': 1}

Input: word_frequency(['a', 'b', 'a', 'a'])
Expected: {'a': 3, 'b': 1}

59. Create a class Student with __init__(self, name, mark) and a method result(self) that returns 'Pass' if mark >= 35 else 'Fail'. (5 Marks)

Starter code:

```
class Student:  
    # write your code here  
    pass
```

Visible test cases:

Input:
s = Student('Alex', 80)
s.result()
Expected: Pass

Input:
s = Student('Bala', 20)
s.result()
Expected: Fail

60. Create a class Rectangle with `__init__`(self, length, breadth) and a method `area(self)` that returns `length * breadth`. (5 Marks)

Starter code:

```
class Rectangle:  
    # write your code here  
    pass
```

Visible test cases:

Input:

```
r = Rectangle(5, 4)  
r.area()
```

Expected: 20

Input:

```
r = Rectangle(10, 2)  
r.area()
```

Expected: 20